

RUPTURE

Rapport technique officiel — Version 0 (bêta)

Projet	RUPTURE — Projet 3 (indépendant de MURPHY et FRONTIÈRES)
Version	V0 — bêta smartphone
Date du rapport	12 septembre 2026
Dépôt	scuillerjack-oss/rupture-v0 (public)
Branche	main
Commit final couvert par ce rapport	e96ef53aa313ed263183f83a509de24a5ac5c869
URL publique	https://scuillerjack-oss.github.io/rupture-v0/
Hébergement	GitHub Pages, déployé via GitHub Actions

Ce rapport décrit l'état réel du dépôt et des vérifications réellement exécutées au commit ci-dessus. Il distingue explicitement ce qui est fait/testé/déployé de ce qui est prévu, recommandé ou non réalisé, conformément à la méthode de travail convenue.

1. Ce qui a été réellement construit

RUPTURE V0 est une PWA mobile-first écrite en JavaScript vanilla, hébergée en continu sur une URL stable, avec pipeline de build/tests/déploiement automatisé. Le jeu est un prototype de stratégie/simulation de crise abstraite : le joueur pilote une Anomalie non biologique qui se propage à travers un réseau de 14 territoires fictifs pendant que le monde réagit progressivement.

- Écran de menu, sélection d'origine sur la carte, écran de jeu, écran de fin explicite (victoire/défaite avec raison).
- Carte en graphe SVG de 14 territoires reliés par 20 routes, sélectionnables au tactile.
- Moteur de simulation déterministe séparé du rendu (aucune règle de jeu dans l'UI).
- Ressource Influence, trois familles d'améliorations (Propagation, Résilience, Discrétion) à effets réellement différenciés.
- Réactions locales (conscience, confinement, fermeture de routes) et réaction globale (confinement mondial).
- Conditions de victoire et de défaite explicites, plus un filet de sécurité à 400 jours.
- Sauvegarde locale (localStorage) avec reprise automatique et réinitialisation propre en nouvelle partie.
- PWA installable (manifest, icônes, service worker) sous le sous-chemin /rupture-v0/.
- Suite de 27 tests automatisés (node:test, zéro dépendance) intégrée au pipeline CI avant chaque déploiement.

2. Architecture et technologies choisies

Stack

- JavaScript ES modules, sans framework UI — volume de code volontairement réduit pour une V0.
- Bundler : Vite (build de production, dev server).
- Rendu carte : SVG (graphe de nœuds/routes plutôt qu'une carte géographique réaliste — lisibilité mobile et identité propre).
- Persistance : localStorage (suffisant pour une bêta solo, pas de backend requis en V0).
- Tests : node:test (runner natif de Node.js, aucune dépendance externe à maintenir).

Structure du dépôt

```
src/engine/ (territories.js, balance.js, state.js, simulation.js) – logique pure, sans DOM, testable en isolation.
src/ui/ (map.js, hud.js, screens.js) – rendu, consomme l'état du moteur sans devenir source de vérité.
src/save.js, src/main.js – persistance et boucle principale.
public/ – manifest.webmanifest, icônes, service worker.
tests/ – suite node:test.
.github/workflows/deploy.yml – tests → build → déploiement GitHub Pages.
```

Choix techniques justifiés

- Carte en graphe plutôt que géographique : évite toute ressemblance avec un jeu existant, grosses zones tactiles, propagation lisible sans surcharge visuelle.
- Pas de framework UI : V0 volontairement petite, DOM re-rendu par templates de chaînes — suffisant à cette échelle, pas de dette d'architecture à ce stade.
- GitHub Pages + GitHub Actions plutôt que Vercel/Netlify : gratuit, adossé directement au dépôt (pas de compte tiers à gérer), même URL stable à chaque redéploiement.

- Simulation en pas fixes (un tick = un jour) plutôt qu'en delta-temps continu : comportement déterministe et testable, conforme à l'exigence de contrôlabilité.

3. Fonctionnalités et règles de gameplay réellement actives

Boucle de jeu

- Nouvelle partie → choix tactile du territoire d'origine → l'anomalie y démarre avec une crise initiale.
- Chaque jour (tick) : la crise croît localement, se propage aux voisins connectés dont la crise dépasse un seuil, et la conscience/confinement local suit avec un temps de retard.
- Les territoires dont la conscience dépasse un seuil peuvent fermer une route voisine (probabiliste), ralentissant la propagation vers ce territoire.
- Le confinement mondial (moyenne de conscience) augmente en continu ; la domination mondiale (crise moyenne pondérée par population) aussi.
- Victoire : domination mondiale $\geq 75\%$. Défaite : confinement mondial = 100 % avant, ou filet de sécurité à 400 jours (défaite par défaut).
- L'écran de fin indique explicitement quel seuil a été franchi — jamais de fin sans explication.
- Sauvegarde automatique à chaque tick et à chaque action ; une nouvelle partie réinitialise entièrement l'état sans mélange avec la précédente (vérifié par test).

Économie et choix stratégiques

- Influence gagnée en continu, proportionnellement à la crise pondérée par population des territoires affectés.
- Propagation : augmente la vitesse de propagation vers les voisins (effet +75 % par niveau, 5 niveaux).
- Résilience : réduit l'effet freinateur du confinement local sur la croissance de la crise (+45 %/niveau).
- Discretion : ralentit la prise de conscience mondiale et les fermetures de routes (+42 %/niveau).
- Les trois familles produisent des parties sensiblement différentes (vérifié par simulation, voir section 6).

4. Choix d'équilibrage et itérations réelles

L'équilibrage a été ajusté trois fois pendant ce chantier, chaque fois vérifié par simulation avant d'être retenu :

- Version initiale : l'anomalie gagnait quasi systématiquement même sans aucune amélioration achetée — aucune décision réelle n'était nécessaire. Corrigé.
- Deuxième itération : la défaite est devenue systématique en jeu passif, mais seuls 3 des 14 territoires d'origine (les mieux connectés) permettaient la victoire même avec une stratégie active — origine imposée de fait. Corrigé en renforçant l'effet des améliorations.
- Version retenue : jeu passif perdant à 100 % (14/14), jeu actif gagnant depuis 12 des 14 origines. Voir section 6 pour le détail exact par territoire.

Ce pré-équilibrage technique n'est explicitement pas définitif : le rythme, le plaisir et la difficulté fine restent à valider par la bêta humaine (voir section 12).

5. Tests automatisés — ce qui est exécuté et le résultat réel

Suite **node:test** (aucune dépendance externe), exécutée en local et en CI. Résultat au commit e96ef53 : **27/27 tests passés, 0 échec**.

Domaine	Ce qui est vérifié
Initialisation	État menu propre, territoires à zéro, structure complète.
Propagation	Croissance de la crise à l'origine ; atteinte d'au moins un voisin connecté ; absence de contamination d'un territoire non atteignable.
Économie d'Influence	Toujours ≥ 0 , jamais décroissante seule, sur une partie complète.
Améliorations	Coût déduit exactement, refus si fonds insuffisants, refus au-delà du niveau max, refus d'un type d'amélioration inconnu.
Réactions locales	Conscience et confinement local augmentent avec la crise.
Réaction globale	Confinement mondial croissant, borné [0,100].
Victoire	Déclenchée au seuil de domination, raison de fin renseignée.
Défaite	Déclenchée au seuil de confinement mondial en jeu passif, raison de fin renseignée.
Filet de sécurité	Fin forcée si le jour atteint le plafond (anti-boucle infinie).
Invariants moteur	Aucun NaN, aucune valeur négative, toutes les grandeurs bornées [0,100] sur 250 jours.
Sauvegarde/reprise	Round-trip exact save→load, suppression, rejet d'une version de sauvegarde incompatible.
Nouvelle partie	Aucune contamination de l'état d'une partie précédente.
Monte-Carlo passif	Les 14 origines testées sans achat : défaite systématique (14/14).
Monte-Carlo actif	Les 14 origines testées avec achats systématiques : victoire depuis 12/14 origines (Lyrath et Nyxor restent difficiles avec cette stratégie simple — voir section 4).

Tests End-to-End supplémentaires exécutés en navigateur réel (Chromium via Playwright, hors suite automatisée committée) : boucle complète menu → choix d'origine → partie → écran de fin (victoire et défaite obtenues séparément), sauvegarde/reprise après rechargement de page, zéro erreur console à chaque exécution.

6. Détail des simulations d'équilibrage (Monte-Carlo, 14 origines)

Origine	Jeu passif	Jeu actif (achats systématiques)
Arca	Défaite	Victoire (j.53)
Boréal	Défaite	Victoire (j.47)
Cindra	Défaite	Victoire (j.51)
Delthia	Défaite	Victoire (j.53)
Esser	Défaite	Victoire (j.48)
Fenwick	Défaite	Victoire (j.41)
Gora	Défaite	Victoire (j.45)
Halvern	Défaite	Victoire (j.52)
Ismer	Défaite	Victoire (j.50)
Jotun	Défaite	Victoire (j.44)
Kelsor	Défaite	Victoire (j.47)
Lyrath	Défaite	Défaite (j.54)
Meridia	Défaite	Victoire (j.51)

Nyxor	Défaite	Défaite (j.53)
-------	---------	----------------

Lecture : le choix du point de départ compte réellement dans la difficulté. Lyrath et Nyxor (faible population, seulement deux connexions chacun) restent perdants même avec une stratégie active simple — assumé comme un gradient de difficulté à ce stade, à confirmer ou corriger après la bêta humaine.

7. Problèmes découverts et corrigés pendant ce chantier

- Zones tactiles sous la barre recommandée (35-36px de hauteur) sur les boutons de vitesse et d'amélioration — remontées à 44px minimum, revérifié sans régression sur 360px et 412px de large.
- Équilibrage initial permettant une victoire quasi automatique sans aucune décision du joueur — corrigé (voir section 4).
- Écart de déploiement : la création du dépôt via l'API GitHub a échoué (permission de l'intégration), contournée en demandant la création manuelle du dépôt, puis rattachement à la session.
- Premier déploiement GitHub Pages en échec (404 — Pages non activé sur le dépôt) : diagnostiqué précisément via les logs du job GitHub Actions, corrigé après activation manuelle par l'utilisateur (Settings → Pages → Source : GitHub Actions).

8. Limites connues à la clôture de la V0

- Équilibrage pré-technique, non définitif : Lyrath et Nyxor restent difficiles en stratégie active simple (section 6) ; rythme et plaisir non validés par un humain.
- Une seule origine de sauvegarde (un seul emplacement local) — suffisant pour la bêta solo demandée, pas multi-profils.
- Identité visuelle fonctionnelle mais minimale (couleurs/formes géométriques) — pas d'illustration ni d'identité graphique poussée, volontairement hors périmètre V0.
- Pas de mode hors-ligne complet garanti : le service worker met en cache au fil de la navigation (stratégie réseau-puis-cache) plutôt qu'un pré-cache exhaustif ; un premier chargement nécessite une connexion.
- Aucun compte, aucun classement, aucun contenu additionnel — conforme au périmètre V0 demandé (point 24).
- Limite de vérification de cette session : l'environnement d'exécution de Claude Code applique une politique réseau sortant qui bloque l'accès direct à des domaines arbitraires (dont github.io). Le déploiement a donc été vérifié via la confirmation serveur de GitHub ("Reported success!", voir section 9) et via un build local strictement identique testé de bout en bout, mais pas par un chargement direct de l'URL publique depuis cette session. L'ouverture réelle sur votre téléphone (section 11) est la vérification finale qui manque de mon côté.

9. Méthode et état du déploiement

Chaîne : push sur **main** → GitHub Actions (.github/workflows/deploy.yml) → npm ci → npm test → npm run build → publication de dist/ comme artefact Pages → déploiement GitHub Pages. La même URL est conservée à chaque redéploiement.

Dernière exécution vérifiée (run GitHub Actions #34695110598, tentative 2)

- Job build : succès — npm ci, npm test (27/27), npm run build, upload de l'artefact Pages.
- Job deploy : succès — log GitHub Actions : « Reported success! » puis « Evaluated environment url: <https://scuillerjack-oss.github.io/rupture-v0/> ».
- Commit déployé : e96ef53aa313ed263183f83a509de24a5ac5c869 — identique au commit final de ce rapport.

Un premier essai de déploiement (commit 152abcb) avait échoué avec une erreur 404 explicite indiquant que GitHub Pages n'était pas activé sur le dépôt. Après activation manuelle (Source : GitHub Actions) par l'utilisateur, le redéploiement du commit courant a réussi sans autre changement de code.

10. Configuration et état réel de la PWA

Élément	Statut vérifié
manifest.webmanifest	Chargé (HTTP 200), name/short_name/start_url/scope/display=standalone présents.
start_url / scope	/rupture-v0/ — cohérent avec le sous-chemin GitHub Pages.
Icônes	192×192, 512×512, 512×512 maskable — les trois chargées en 200, type image/png.
Service worker	Enregistré et actif immédiatement après chargement (stratégie réseau-puis-cache).
Chemins d'assets sous /rupture-v0/	Vérifiés via base Vite dédiée — aucun chemin absolu cassé.
Erreurs console critiques	Aucune, sur l'ensemble des parcours de test automatisés en navigateur réel.
Mise à jour de version	Le service worker recharge depuis le réseau à chaque requête (pas de cache figé) ; un nouveau déploiement est donc visible sans étape manuelle de purge.

Toutes les vérifications ci-dessus ont été faites sur le build de production exact (dist/) déployé, servi localement dans les mêmes conditions (même base /rupture-v0/), et non sur le serveur de développement.

11. Procédure de bêta smartphone (Android)

- Ouvrir Chrome sur le téléphone Android.
- Aller sur : <https://scuillerjack-oss.github.io/rupture-v0/>
- Appuyer sur le menu (trois points, en haut à droite) de Chrome → « Installer l'application » (ou « Ajouter à l'écran d'accueil » selon la version de Chrome).
- Confirmer l'installation : une icône RUPTURE apparaît sur l'écran d'accueil, comme une application native.
- Lancer RUPTURE depuis cette icône (pas depuis le navigateur) pour la bêta.
- Appuyer sur « Nouvelle partie », toucher un territoire pour y implanter l'anomalie, puis « Commencer ».
- Jouer : observer la propagation, acheter des améliorations avec l'Influence accumulée, ajuster la vitesse/pause.
- Fermer complètement l'application puis la rouvrir : la partie doit reprendre exactement où elle en était (sauvegarde locale).
- Jouer jusqu'à un écran de fin (victoire ou défaite) et vérifier que la raison affichée est compréhensible, puis recommencer une partie.

Si l'installation échoue ou ne propose qu'un simple raccourci sans mode standalone, c'est un point de blocage réel à me signaler : ce n'est pas équivalent à une PWA installable et devra être corrigé avant de considérer la bêta valide.

12. Points explicitement laissés à la bêta humaine

- Rythme et plaisir de la boucle de jeu (durée moyenne d'une partie, sensation de progression).
- Difficulté perçue selon l'origine choisie, en particulier Lyrath et Nyxor (section 6).
- Clarté réelle des réactions du monde pour un joueur qui découvre le jeu sans explication orale.
- Pertinence de la vitesse de propagation / du rythme des fermetures de routes sur petit écran, en conditions réelles (pas seulement en navigateur automatisé).
- Confirmation que l'installation PWA fonctionne comme attendu sur votre appareil Android réel et votre version de Chrome.
- Identité visuelle et habillage : la V0 est fonctionnelle mais volontairement non stylisée graphiquement.

13. État exact du projet à la clôture de ce chantier

Critère (cahier des charges, point 25)	État
A. Démarre depuis l'URL publique	Confirmé côté serveur GitHub ; à confirmer par vous sur votre téléphone (section 8, limite réseau de session).
B. Partie complète jouable	Oui — vérifié en navigateur réel, victoire et défaite obtenues.
C. Propagation fonctionne	Oui — testé automatiquement et manuellement.
D. Influence fonctionne	Oui — testé (jamais négative, toujours dépensable).
E. Propagation/Résilience/Discrétion produisent des choix	Oui — Monte-Carlo : 0/14 victoires passives vs 12/14 actives.
F. Le monde réagit	Oui — conscience, confinement local, fermetures de routes, confinement mondial actifs.

Critère (cahier des charges, point 25)	État
G. Victoire et défaite fonctionnent	Oui — avec raison explicite affichée.
H. Sauvegarde/reprise fonctionnent	Oui — testé automatiquement (round-trip) et manuellement (rechargement navigateur).
I. Interface exploitable sur smartphone	Oui — zones tactiles $\geq 44\text{px}$, testé sans débordement à 360px et 412px.
J. PWA techniquement installable	Critères techniques tous vérifiés (section 10) ; installation réelle sur votre appareil à confirmer.
K. Build de production valide	Oui — vérifié à chaque itération, dernière fois au commit e96ef53.
L. Tests critiques passent	Oui — 27/27 en local et en CI GitHub Actions.
M. Déploiement permanent fonctionne	Oui — confirmé par les logs serveur GitHub Actions (section 9).
N. main correspond à la version déployée	Oui — même commit e96ef53 des deux côtés.
O. Rapport technique PDF existe	Ce document, committé dans le dépôt sous docs/.

Seul le contact final entre l'URL publique et votre téléphone (critères A et J) n'a pas pu être vérifié personnellement par Claude, pour une raison d'infrastructure de session expliquée en section 8 — tout le reste de la chaîne technique est vérifié et documenté ci-dessus.